RAPTOR DEFENSE V2.0 // HANDBOOK

RAPTOR DEFENSE // CAHIER DES CHARGES TECHNIQUE & SPÉCIFICATIONS OPÉRATIONNELLES

Sous-Système d’Urgence & Mode TTY Dégradé : Utilitaires `rap` & `rapless`

Document de Référence Architecturale — Révision 1.0 (Post-v2.0)

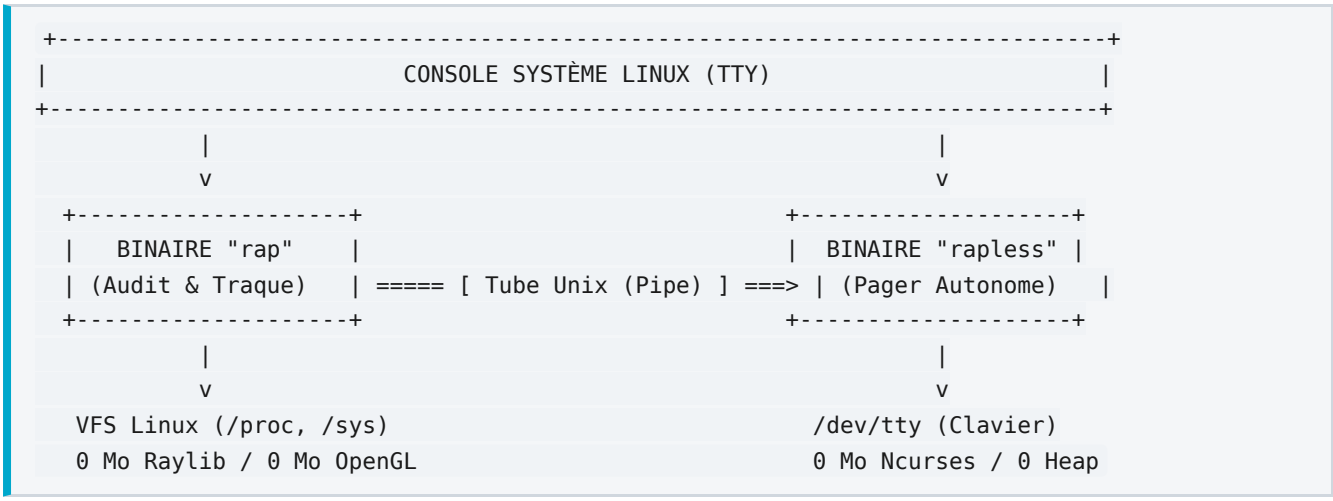
---

1. CONTEXTE OPÉRATIONNEL & DOCTRINE D'ENGAGEMENT

Le sous-système CLI d'urgence de RAPTOR est conçu pour intervenir dans des contextes d'hostilité extrême où le système d'exploitation Linux subit une dégradation critique : * Saturation agressive de la mémoire vive (*Out Of Memory / OOM starvation*). * Défaillance ou absence totale du serveur d'affichage graphique (X11 / Wayland indisponible, mode *single-user*, console TTY série ou session SSH d'urgence). * Présence d'implants malveillants déployant des techniques de déni de service local ou de dissimulation de processus.

Dans ces conditions, l'interface graphique souveraine (Raylib 5.0) ne peut pas ou ne doit pas être instanciée. Le sous-système CLI sépare physiquement les capacités d'audit et d'intervention en deux micro-binaires autonomes :

1. `rap` **(RAPTOR Tactical Investigator) :** Outil de diagnostic instantané, d'extraction forensique et de neutralisation cinétique. Il réutilise le cœur d'audit modulaire (`src/core/`) sans lier aucune bibliothèque graphique.
2. `rapless` **(RAPTOR Emergency Pager) :** Téléscripteur et visualiseur paginé interactif ultra-léger, indépendant de `libncurses` et de tout composant externe, conçu pour paginer les sorties de `rap` ou n'importe quel fichier système en cas de sinistre.



---

2. CONTRAINTES ARCHITECTURALES & MÉMOIRE STRICTES

Les deux binaires sont assujettis aux impératifs d'ingénierie suivants :

1. **Empreinte Mémoire Résidente (RSS)** < 2 Mo **:** Les binaires doivent s'exécuter dans un environnement de famine mémoire sans déclencher l'OOM-Killer du noyau.
2. **Allocation Dynamique Proscrite sur le Chemin Critique :** Aucun appel à `malloc()` ou `calloc()` n'est toléré dans `rapless`. Les tampons sont statiques et alloués dans le segment `.bss` ou sur la pile d'exécution (*stack*).
3. **Zéro Dépendance Graphique :** Interdiction formelle de lier `libraylib`, `libGL`, `libm` ou `libX11`. Seule la `libc` POSIX standard est autorisée, avec compatibilité garantie pour la compilation statique pure (`gcc -static` via `musl-libc` ou `glibc`).
4. **Composabilité Unix Totale :** Détection dynamique du terminal via `isatty(STDOUT_FILENO)`. En mode interactif, affichage enrichi ANSI (couleurs défensives). En mode pipé ou redirigé, émission de flux brut tabulaire (TSV / *raw text*) sans codes d'échappement pour parsing direct par `grep`, `awk`, `cut`, `xargs`.
5. **Codes de Sortie POSIX Standardisés :**
   - Code `0` : Système sain / Aucune anomalie détectée / Commande exécutée avec succès.
   - Code `1` : **Menace active confirmée** (permet l'usage en sentinelle automatisée : `rap hunt || alert`).
   - Code `2` : Erreur de syntaxe, paramètre invalide ou permissions `root` insuffisantes.

---

3. SPÉCIFICATION DE L'UTILITAIRE TACTIQUE `RAP`

3.1. Grammaire & Syntaxe d'Appel

```
rap [COMMANDE] [OPTIONS] [ARGUMENTS]
```

Drapeaux Universels

Drapeau Court	Drapeau Long	Description Opérationnelle
`-q`	`--quiet`	Mode silencieux : supprime en-têtes et décorations. N'émet que les identifiants cibles (ex: liste de PID bruts).
`-r`	`--raw`	Format machine : désactive les unités dynamiques (Mo/Ko) au profit d'octets bruts délimités par des tabulations.
`-v`	`--verbose`	Mode forensique détaillé : affiche chemins complets, PPID, UID et ligne de commande brute `/proc/[PID]/cmdline`.
`-f`	`--force`	Contourne les demandes de confirmation lors des actions cinétiques (`kill` , `freeze`).
`-n <N>`	`--limit <N>`	Limite la sortie aux *N* premiers éléments qualifiés (par défaut : 5 pour `top`).

3.2. Matrice des Sous-Commandes `rap`

1. `rap status` (Alias : `rap st`)

- **Rôle :** Synthèse d'évaluation vitale exécutée en moins de 20 millisecondes.
- **Données présentées :**
  - Charge CPU instantanée différentielle.
  - Mémoire vive réelle : disponible vs totale (`MemAvailable` issue de `/proc/meminfo`).
  - Volume de processus scannés vs processus suspects.
  - État d'intégrité de `/etc/ld.so.preload` .
  - Statut d'alerte global (`NOMINAL` vert ou `COMPROMISSION ACTIVE` rouge).
- **Code retour :** `0` si aucune menace, `1` si au moins une menace est active.

2. `rap top [-n N]`

- **Rôle :** Tri décroissant des plus gros consommateurs de mémoire résidente (`VmRSS`).
- **Traitement spécifique :** Purge automatique des threads du noyau (`rss == 0` et binaires pointant vers `[système/restreint]`).
- **Sortie standard :** `PID | NOM | RAM (RSS) | DÉRIVE | CHEMIN DISQUE`.

3. `rap hunt [-q] [-v]`

- **Rôle :** Scan combiné d'ensemble à haute vélocité.
- **Corrélation :** Déclenche simultanément l'évaluation des mutations anonymes R-X (Section 13), l'audit des variables `LD_PRELOAD` (Section 14) et les binaires exécutés dissociés du disque (`(deleted)`).
- **Sortie standard :** Ne liste que les processus présentant une anomalie active. Si le poste est intègre, affiche une confirmation verte en une ligne et sort avec le code `0`.

4. `rap rx [-q] [-v]`

- **Rôle :** Traque chirurgicale des mutations de mémoire virtuelle (Section 13).
- **Algorithme :** Analyse `/proc/[PID]/maps` pour isoler les segments cumulant permissions `r-xp`, Inode `0` et absence de descripteur de fichier adossé.
- **Filtre :** Confrontation systématique au registre souverain d'exclusion `mem_whitelist.conf` pour éliminer les compilateurs JIT légitimes (Chromium, Firefox, Flatpak).
- **Sortie standard :** `PID | EXÉCUTABLE | PLAGE MÉMOIRE | TAILLE (Ko) | STATUT`.

5. `rap preload [-q] [-v]`

- **Rôle :** Traque exclusive du chargeur dynamique et des rootkits userland (Section 14).
- **Algorithme :** Audit passif direct de `/etc/ld.so.preload` et extraction des blocs d'environnement `/proc/[PID]/environ` découpés sur l'octet nul (`\0`).
- **Sortie standard :** `PID | EXÉCUTABLE | VECTEUR | BIBLIOTHÈQUE INJECTÉE`.

6. `rap net [-q] [-v] [-n N]`

- **Rôle :** Audit des communications réseau actives et corrélation Net-Guard (Section 11).
- **Algorithme :** Rapprochement triangulaire `/proc/net/{tcp,udp}` ↔ `/proc/[PID]/fd/*`.
- **Mise en relief critique :** Signalement rouge prioritaire pour tout processus disposant d'un socket ouvert tout en hébergeant des plages de mémoire modifiables/exécutables (RWX).
- **Sortie standard :** `PID | EXÉCUTABLE | PROTO | ADRESSE LOCALE | ADRESSE DISTANTE | ÉVALUATION`.

7. `rap kill [-f] <PID>`

- **Rôle :** Neutralisation cinétique chirurgicale immédiate par signal `SIGKILL` direct.
- **Gardes-fous d'immunité système :**
  - Refus catégorique et immédiat d'abattre le PID `1` (`init` / `systemd`).
  - Refus d'abattre le PID de `rap` ou le PID du shell parent de l'opérateur (`$PPID`).
  - Refus d'abattre les threads internes du noyau.
- **Contre-vérification TOCTOU (*Time-Of-Check to Time-Of-Use*) :**
  - Relit `/proc/[PID]/exe` immédiatement avant l'émission de `kill(PID, SIGKILL)` pour s'assurer que le processus n'a pas été recyclé par le noyau entre la détection et la frappe.

8. `rap freeze [-f] <PID>`

- **Rôle :** Gel cinétique immédiat via le signal non interceptable `SIGSTOP`.
- **Objectif forensique :** Fige l'exécution d'un implant furtif pour bloquer une fuite de données ou une charge destructive tout en préservant l'intégrité de sa mémoire vive pour génération d'un core dump (`gcore`) ou extraction du binaire.

9. `rap man [COMMANDE]`

- **Rôle :** Affichage de la documentation opérationnelle intégrée.
- **Persistance immuable :** Le texte complet des pages d'aide est compilé directement dans la section `.rodata` de l'exécutable `rap`, garantissant son accessibilité même si `/usr/share/man/` est détruit ou inaccessible.

---

4. SPÉCIFICATION DU TÉLÉSCRIPTEUR AUTONOME `RAPLESS`

4.1. Mission & Philosophie de Conception

`rapless` est un visualiseur de texte paginé autonome de secours. Il n'a qu'une seule mission : afficher un flux textuel page par page, permettre d'avancer, de reculer et de quitter proprement, sans jamais corrompre l'état du terminal.

- **Zéro librairie externe :** Aucune utilisation de `ncurses`, `termlib` ou `tinfo`.
- **Zéro allocation dynamique sur le tas :** Zéro appel à `malloc()`.
- **Architecture à double flux :**
  - Le flux de données est lu sur `stdin` (pipe) ou depuis un fichier régulier (`argv[1]`).
  - Les interactions clavier sont **toujours** lues directement sur `/dev/tty`.

```
       [ Flux Données : STDIN (Pipe) OU Fichier ]
                           |
                           v
               +-------------------+
               |  MOTEUR rapless   | <=== [ Contrôle Clavier : /dev/tty ]
               +-------------------+
                           |
                           v
              [ Écran TTY : STDOUT ]
```

4.2. Ergonomie & Jeu de Commandes Clavier

Le terminal est configuré en mode non-canonique sans écho via `termios`. Le moteur réagit instantanément à l'octet brut :

Touche	Action Opérationnelle	Comportement Détaillé
`Espace`	**Page Suivante**	Fait défiler d'une hauteur d'écran complète moins une ligne de contexte.
`Entrée`	**Ligne Suivante**	Fait défiler le flux d'une seule ligne vers le bas.
`b`	**Page Précédente**	Recule d'une page complète dans le tampon de garde circulaire.
`q` ou **`Échap`**	**Quitter**	Restaure immédiatement le terminal et rend la main au prompt du shell.

4.3. Gestion Mémoire & Tampon Circulaire

Pour supporter le défilement arrière sans allouer de mémoire dynamique : * `rapless` maintient un tampon circulaire statique de garde fixé à 64 Ko (`RAPLESS_BUFFER_SIZE`). * Pour la lecture de fichiers sur disque, `rapless` utilise la lecture par blocs POSIX `read()` ou la projection `mmap(PROT_READ)` . * En fin d'écran, une barre d'état inversée monochrome affiche la progression : `text -- RAPLESS // [Espace: Suivant | b: Retour | Entrée: Ligne | q: Quitter] --`

4.4. Préservation de l'Intégrité du Terminal (Robustesse de Sortie)

Afin d'éviter de laisser la console de l'administrateur en mode aveugle (écho coupé) en cas d'interruption abrupte : 1. Les attributs originaux du terminal (`struct termios orig_termios`) sont sauvegardés dès l'initialisation. 2. La fonction de restauration `rapless_restore_terminal()` est enregistrée auprès de la libc via `atexit()` . 3. Un intercepteur de signaux POSIX minimaliste (`sigaction`) capture `SIGINT` (`Ctrl+C`), `SIGTERM` et `SIGWINCH` (redimensionnement dynamique de la console).

---

5. DICTIONNAIRE GLOBAL DES NOUVELLES FONCTIONS (88 À 106)

Le dictionnaire officiel de gouvernance architecturale de RAPTOR est incrémenté de 19 nouvelles fonctions, réparties entre la **Section 16** (`rap`) et la **Section 17** (`rapless`).

```
                        MATRICE DES 17 SECTIONS DE RAPTOR

[ Sec 01-10 : Socle Universel, Forensique RWX & Rapports ] ----> Fonctions 01 à 57
[ Sec 11    : Net-Guard & Filtrage Souverain Sockets ]       ----> Fonctions 58 à 70
[ Sec 12    : Net-Intel & Renseignement BGP/ASN Team Cymru ] ----> Fonctions 71 à 79
[ Sec 13    : Mem-Stealth & Traque Mutations R-X Inode 0 ]   ----> Fonctions 80 à 83
[ Sec 14    : Preload-Hunter & Audit Chargeur Dynamique ]   ----> Fonctions 84 à 85
[ Sec 15    : Notifications Asynchrones Freedesktop ]        ----> Fonctions 86 à 87
[ Sec 16    : Moteur Tactique d'Urgence CLI (rap) ]          ----> Fonctions 88 à 99
[ Sec 17    : Téléscripteur & Pager Autonome (rapless) ]     ----> Fonctions 100 à 106
```

5.1. Section 16 — Moteur Tactique d'Urgence CLI (`src/cli/rap_main.c`)

88. `rap_cli_entry_point`

- **Rôle :** Point d'entrée principal (`main`) de l'exécutable `rap` .
- **Responsabilité :** Détecte la présence d'arguments, initialise les descripteurs de capture et oriente l'exécution vers le parseur de sous-commandes.

89. `rap_cli_parse_args`

- **Rôle :** Analyseur lexical des drapeaux d'action CLI (`-q` , `-r` , `-v` , `-f` , `-n`).
- **Responsabilité :** Normalise les options transmises et configure la structure globale de contexte d'exécution `rap_options_t` .

90. `rap_cmd_status`

- **Rôle :** Exécuteur de la sous-commande `rap status` .
- **Responsabilité :** Interroge en séquence ultra-rapide `/proc/stat` , `/proc/meminfo` et `/etc/ld.so.preload` pour calculer le bilan de santé immédiat en moins de 20 ms.

91. `rap_cmd_top`

- **Rôle :** Exécuteur de la sous-commande `rap top` .
- **Responsabilité :** Parcourt `/proc` , extrait `VmRSS` , élimine les threads système non résidents (`rss == 0`) et affiche les *N* plus gros consommateurs.

92. `rap_cmd_hunt`

- **Rôle :** Exécuteur de la commande universelle `rap hunt` .
- **Responsabilité :** Déclenche les sondes combinées Sections 13, 14 et binaires supprimés. N'affiche que les alertes positives.

93. `rap_cmd_rx`

- **Rôle :** Exécuteur de la sous-commande `rap rx` (Section 13).
- **Responsabilité :** Analyse `/proc/[PID]/maps`, applique le filtre `mem_whitelist.conf` et liste les pages orphelines Inode 0 devenues exécutables via `mprotect()`.

94. `rap_cmd_preload`

- **Rôle :** Exécuteur de la sous-commande `rap preload` (Section 14).
- **Responsabilité :** Audite la persistance globale `/etc/ld.so.preload` et inspecte les chaînes `LD_PRELOAD` dans `/proc/[PID]/environ`.

95. `rap_cmd_net`

- **Rôle :** Exécuteur de la sous-commande `rap net` (Section 11).
- **Responsabilité :** Résout la triangulation Socket ↔ Inode ↔ PID et met en relief critique les processus réseau porteurs de segments RWX.

96. `rap_cmd_kill`

- **Rôle :** Exécuteur de la sous-commande cinétique `rap kill`.
- **Responsabilité :** Vérifie l'éligibilité de la cible via `rap_safety_check_pid()`, valide l'absence de recyclage de PID et transmet le signal `SIGKILL`.

97. `rap_cmd_freeze`

- **Rôle :** Exécuteur de la sous-commande d'isolation `rap freeze`.
- **Responsabilité :** Vérifie les gardes-fous immunitaires et suspend immédiatement le processus cible via le signal `SIGSTOP`.

98. `rap_cmd_man`

- **Rôle :** Exécuteur du manuel embarqué `rap man`.
- **Responsabilité :** Localise le bloc d'aide requis dans la section `.rodata` et le projette sur `stdout`.

99. `rap_safety_check_pid`

- **Rôle :** Barrière de sécurité et de contrôle d'immunité des processus.
- **Responsabilité :** Interdit formellement toute action destructrice sur le PID 1, les threads noyau et les ancêtres du shell opérateur.

5.2. Section 17 — Téléscripteur & Pager Autonome (`src/cli/rapless.c`)

100. `rapless_entry_point`

- **Rôle :** Point d'entrée principal (`main`) de l'exécutable `rapless`.
- **Responsabilité :** Détermine la source du flux (descripteur `0` ou fichier passé en argument), initialise le terminal et transfère le contrôle à la boucle d'événements.

101. `rapless_init_terminal`

- **Rôle :** Bascule du terminal en mode non-canonique d'urgence.
- **Responsabilité :** Sauvegarde la configuration active via `tcgetattr()` et applique les drapeaux `~ICANON` et `~ECHO` via `tcsetattr()`. Ouvre `/dev/tty` pour écouter le clavier.

102. `rapless_restore_terminal`

- **Rôle :** Restauration garantie des paramètres originaux du terminal.
- **Responsabilité :** Réapplique la structure `termios` sauvegardée et repositionne le curseur standard. Enregistrée via `atexit()`.

103. `rapless_get_window_size`

- **Rôle :** Détection de la géométrie de la console.
- **Responsabilité :** Interroge `ioctl(STDOUT_FILENO, TIOCGWINSZ, &ws)`. En cas d'échec ou d'exécution sur terminal dégradé, applique les valeurs de secours par défaut : 80 colonnes $\times$ 24 lignes.

104. `rapless_feed_from_fd`

- **Rôle :** Remplissage du tampon de données paginé.
- **Responsabilité :** Lit les octets depuis le descripteur d'entrée dans le tampon circulaire de 64 Ko sans aucune allocation dynamique sur le tas.

105. `rapless_render_page`

- **Rôle :** Dessin d'une page écran et de la barre d'état.
- **Responsabilité :** Émet le nombre exact de lignes correspondant à la hauteur détectée du terminal et affiche la ligne de navigation inférieure.

106. `rapless_event_loop`

- **Rôle :** Boucle de contrôle interactive et écoute du clavier.
- **Responsabilité :** Lit les frappes bloquantes sur `/dev/tty`, intercepte `Espace`, `Entrée`, `b`, `q` et déclenche le rafraîchissement ou la sortie immédiate.

---

6. INTÉGRATION AU SYSTÈME DE CONSTRUCTION (`CMAKELISTS.TXT`)

L’intégration dans le projet respecte le principe de découplage absolu. Les cibles `rap` et `rapless` sont compilées indépendamment du moteur graphique principal :

```
# ================================================================================
# CIBLES DE SECOURS CLI SOUVERAINES (SANS RAYLIB / SANS OPENGL)
# ================================================================================

# 1. Binaire tactique d'investigation et neutralisation "rap"
add_executable(rap
    src/cli/rap_main.c
    src/core/mem_stealth.c
    src/core/preload_hunter.c
    src/core/raptor_registry.c
    src/core/raptor_differential.c
)
target_include_directories(rap PRIVATE include src/core)
target_compile_definitions(rap PRIVATE RAPTOR_CLI_MODE=1)

# 2. Binaire téléscripteur / pager autonome "rapless"
add_executable(rapless
    src/cli/rapless.c
)
target_include_directories(rapless PRIVATE include)

# Installation directe dans l'arborescence binaire système
install(TARGETS rap rapless DESTINATION /usr/local/bin)
```

---

7. CRITÈRES DE VALIDATION & TESTS D’ACCEPTATION

Pour que le sous-système CLI soit déclaré apte au service opérationnel, il doit franchir avec succès les épreuves suivantes :

1. **Épreuve d’OOM Starvation :** Exécution de `rap status` et `rap hunt` sur une machine dont la mémoire disponible est inférieure à $50\ \text{Mo}$ sans déclenchement de l’OOM-Killer.
2. **Épreuve de Survie du Terminal (`Ctrl+C`) :** Interruption forcée par signal `SIGINT` au milieu d’une session `rapless` : le terminal doit retrouver immédiatement son écho et son mode canonique sans nécessiter l’appel à la commande `reset`.
3. **Épreuve du Pipe Saturé :** Exécution de la commande `cat /dev/urandom | head -c 10M | rapless` : `rapless` doit maintenir sa consommation mémoire strictement sous la barre des $64\ \text{Ko}$ et répondre instantanément à la touche `q` sur `/dev/tty`.
4. **Épreuve de Robustesse Anti-Collision de PID :** Tentative d’abattage d’un processus éteint simulé : `rap kill` doit constater la disparition ou la mutation du descripteur et refuser de tirer sur un nouveau processus ayant hérité du PID.